INTERVIEW REFERENCE

# System Design for Beginners — Complete Notes

Condensed, information-dense notes covering the full breadth of system design fundamentals needed for interviews.

# 1. Why System Design

Toy architecture = `client → backend (NodeJS) → DB`. Fine for prototypes. At millions/billions of users it breaks. Real systems must add: **scaling, fault tolerance, security, monitoring, reliability**.

"Client" = anything a user touches: React app, Android, iOS, desktop.

# 2. What is a Server

A **server** is a physical/virtual machine running your application code.

Request lifecycle for `https://abc.com`:

1. **DNS resolver** maps domain → IP address (unique logical address per machine).
2. Browser opens connection to that IP.
3. Server routes to the right process using the **port** (HTTPS default 443, HTTP 80).

So `https://abc.com` ≡ `35.154.33.64:443`. Domains exist because IPs are unmemorable.

**Deployment** = moving your code onto a cloud provider's VM with a public IP attached (AWS **EC2**, Azure VM, GCP Compute Engine). Renting beats self-hosting because the provider handles power, network, hardware.

# 3. Latency and Throughput

| Metric | Definition | Unit |
|---|---|---|
| **Latency** | Time for ONE request to complete (client → server → back) | ms |
| **Throughput** | Number of requests/units of work handled per unit time | RPS / TPS |
| **RTT (Round Trip Time)** | Full request + response travel time; often used as a synonym for latency | ms |

Analogy: latency = how long one car takes to drive the route (10 min); throughput = how many cars the highway moves per hour (1,000).

Goal: **low latency + high throughput**. Every server has a throughput ceiling; exceeding it causes choking or crashes.

# 4. Scaling and Its Types

Scaling = increase machine specs, or add machines, to absorb load.

### Vertical Scaling (Scale Up/Down)

Increase RAM/CPU/storage on the **same** machine.

- Simple — no code changes, no distribution problems.
- Preferred for **SQL databases** and **stateful applications** (state consistency is hard to distribute).
- Hard ceiling: you cannot grow one box forever.

### Horizontal Scaling (Scale Out/In)

Add **more machines** and distribute load across them.

- Clients can't be given N different IPs — they don't know your topology.
- Solution: put a **load balancer** in front. All clients hit the LB; the LB routes to the least-busy machine.
- This is what's used most of the time in the real world.

## 5. Auto Scaling

Traffic is not constant. Running peak capacity (e.g., 100 EC2s) 24/7 wastes money when you only need 10.

**Auto Scaling** = dynamically change the number of servers based on live metrics. E.g., if average CPU crosses a threshold (say 90%), launch another instance automatically; scale back in when load drops.

Find the real threshold via **load testing**, not guesswork.

## 6. Back-of-the-Envelope Estimation

Spend ~5 minutes on this in an interview — not more. Approximate aggressively to keep math easy.

### Powers table (memorize)

| Power of 2 | Approx value | Power of 10 | Unit |
|---|---|---|---|
| 10 | 1 Thousand | 3 | 1 KB |
| 20 | 1 Million | 6 | 1 MB |
| 30 | 1 Billion | 9 | 1 GB |
| 40 | 1 Trillion | 12 | 1 TB |
| 50 | 1 Quadrillion | 15 | 1 PB |

Estimate three things: **Load**, **Storage**, **Resources**.

### Load estimation (Twitter example)

- Ask for **DAU**. Say 100M DAU, 10 tweets/user/day.
- Writes = 100M × 10 = **1 billion tweets/day**.
- Reads: 1000 tweets read/user/day → 100M × 1000 = **100 billion reads/day**.

### Storage estimation

- Tweet = 200 chars × 2 bytes = 400 B ≈ **500 B** (round to 1000 B for easy math).
- 10% of tweets carry a 2 MB photo → 100M photo tweets.
- Total/day = (1000 B × 1B tweets) + (2 MB × 500M) ≈ 1 TB + 1 PB ≈ **1 PB/day** (1 TB is noise).

### Resource estimation

- 10,000 RPS, 10 ms CPU per request → 10,000 × 10 ms = **100,000 ms of CPU per second**.
- Each core delivers 1000 ms of processing/second → 100,000 / 1000 = **100 cores**.
- 4 cores/server → **25 servers** behind a load balancer.

## 7. CAP Theorem

Applies to **distributed systems** (data on multiple servers). One server in the set = a **node**. Data is replicated across nodes.

Why distribute: spread workload for higher aggregate throughput; place data near users for lower access latency.

- **C — Consistency**: every read returns the same, most-recent result regardless of which node serves it.
- **A — Availability**: system keeps answering requests even when some **nodes fail**.
- **P — Partition Tolerance**: system keeps operating during a **network partition** (nodes can't talk to each other).

> *Availability = survives node failure. Partition tolerance = survives network failure.*

**Theorem**: you can guarantee at most **two of three** simultaneously. CA, AP, CP are each possible; CAP is not.

**In practice P is mandatory** — networks partition. So the real choice is **CP vs AP**.

Why: nodes A, B, C; B gets partitioned off.

- Choose **AP**: keep serving. B can't propagate its writes → users on B see different data than users on A/C. Consistency sacrificed.
- Choose **CP**: refuse requests until the partition heals so all nodes stay identical. Availability sacrificed.

**Choosing:**

- **CP** — banking, payments, stock trading. Stale/inconsistent data is unacceptable.
- **AP** — social media. A briefly wrong `likeCount` is fine.

## 8. Scaling a Database

Scale **incrementally** — don't build for 10M users when you have 10K. That's over-engineering. Apply these in order.

## 8.1 Indexing

Without an index: **full table scan**, O(N) per lookup. With an index: DB copies the column into a **B-tree** (sorted), enabling binary-search-like lookup in **O(log N)**. One line of SQL; the DB handles B-tree maintenance. Costs write overhead and storage.

## 8.2 Partitioning

Split one big table into several smaller tables **on the same DB server** ( `user_table_1/2/3` ).

- Benefit: each partition has its own smaller index → faster index searches once one index file grows huge.
- The DB engine (e.g., PostgreSQL) routes `SELECT * FROM users WHERE id=4` to the right partition automatically; you can also route at the application level.

## 8.3 Master-Slave Architecture (replication)

Use when indexing + partitioning + vertical scaling are exhausted, on **read-heavy** traffic.

- **Master node**: handles all writes (INSERT/UPDATE/DELETE).
- **Slave nodes**: handle reads (SELECT); reads go to the least-busy slave.
- Writes land on master then replicate to slaves **synchronously or asynchronously** (config choice).

## 8.4 Multi-Master Setup

Use when a single master can't absorb the write volume. Multiple masters accept writes and sync periodically.

- Common pattern: geographic — North India master + South India master, each serving its region, replicating to the other.
- **Hardest part: conflict resolution.** Same ID written differently in two masters → you must define the rule in code (last-write-wins, merge, concatenate, keep both). No universal rule; it's a business decision.

## 8.5 Database Sharding

Like partitioning, but partitions live on **different servers** (each called a **shard**). The column used to route is the **sharding key**.

- Sharding key must distribute data **evenly** to avoid hotspots.
- Each shard is independently scalable (e.g., add master-slave to just the hot shard).

**Sharding strategies:**

| Strategy | How | Pro | Con |
|---|---|---|---|
| **Range-based** | Shard 1: id 1–1000, Shard 2: 1001–2000... | Simple | Skewed data → uneven load |
| **Hash-based** | `HASH(user_id) % num_shards` | Even distribution | Rebalancing on shard add is painful (hashes shift) |
| **Geographic / entity-based** | Shard by region/department | Great for geo-distributed systems | Hotspot shards |
| **Directory-based** | Lookup table maps key → shard | Flexible reassignment without app changes | Directory becomes a bottleneck / SPOF |

**Disadvantages of sharding:**

1. Routing logic lives in **application code** — you must know which shard to read/write.
2. **Cross-shard JOINs** require pulling data from multiple servers — expensive.
3. **Consistency is lost/harder** since data spans servers.

### Database scaling rules of thumb

- Always try **vertical scaling first** — cheapest in complexity.
- **Read-heavy** → master-slave replication.
- **Write-heavy** → sharding (data won't fit one machine); avoid cross-shard queries.
- Read-heavy where master-slave is saturated → sharding too, but only at very large scale.

## 9. SQL vs NoSQL

### SQL

- Tables, rows, columns; **predefined/rigid schema** declared before insert.
- **ACID** guarantees → data integrity and reliability.
- Examples: MySQL, PostgreSQL, Oracle, SQL Server, SQLite.

### NoSQL — four families

| Type | Stores | Examples |
|---|---|---|
| **Document** | JSON/BSON documents | MongoDB |
| **Key-value** | key → value pairs | Redis, DynamoDB |
| **Column-family** | data by columns not rows | Cassandra |
| **Graph** | nodes + relationships (mutual friends, friends-of-friends) | Neo4j |

- **Flexible schema** — add fields not in the original design.
- Does not strictly follow ACID; trades it for **scalability and performance**.

### Scaling difference

- SQL: designed to scale **vertically**.
- NoSQL: designed to scale **horizontally**; sharding is native.
- Sharding SQL is possible but usually avoided — you chose SQL for ACID, and consistency + cross-shard JOINs get expensive and complex.

### Which to pick

| Use NoSQL when | Use SQL when |
|---|---|
| Unstructured data / flexible schema (reviews, recommendations) | Structured data with fixed schema (customer accounts) |
| Need high availability, low latency, huge volume that won't fit one server (posts, likes, comments, messages, real-time driver locations) | Need integrity/consistency via ACID (financial transactions, account balances, orders, payments, stock trading) |
| | Complex queries, JOINs, aggregations — e.g., analytics |

## 10. Microservices

- **Monolith**: whole app (users, products, orders, payments) is one deployable backend.
- **Microservices**: independently deployable services, one per business capability — User Service, Product Service, Order Service, Payment Service, each its own backend.

**Why split:**

1. **Independent scaling** — scale only the hot service.
2. **Tech-stack freedom** — User Service in Node, Order Service in Go.
3. **Fault isolation** — Order Service crashing doesn't take down User/Product. In a monolith, one crash kills everything.

**When to use:**

- "Microservices of a startup mirror its internal team structure" — 3 teams → ~3 services; services split as teams grow.
- Most startups **start as a monolith** (2–3 engineers) and migrate as team count grows.
- Also when avoiding single points of failure.

**Client access — API Gateway:** services live on different IPs/hosts; exposing all of them to clients is unmanageable. Clients hit **one endpoint** — the API Gateway — which maps each request to the right microservice. Each service scales independently behind it (Product ×3, User ×2, Payment ×1).

API Gateway extras: **rate limiting, caching, authentication/authorization**.

# 11. Load Balancer Deep Dive

**Why**: single point of contact for clients. Clients hit the LB's domain; the LB forwards to the least-busy backend.

### Algorithms

1. **Round Robin** — requests distributed sequentially in a circle (S1, S2, S3, S1, ...).
   - ➕ Simple; good when servers have equal capacity. ➖ Ignores server health/actual load.
2. **Weighted Round Robin** — servers get weights by capacity; higher weight = more requests (a bigger box gets 2× the traffic).
   - ➕ Handles heterogeneous servers. ➖ Static weights don't reflect real-time performance.
3. **Least Connections** — route to the server with fewest active connections (HTTP/TCP/WebSocket).
   - ➕ Dynamic, based on real activity. ➖ Poor when connection durations vary widely.
4. **Hash-Based** — hash client IP / user_id → server. Same client consistently hits the same server.
   - ➕ Session persistence (sticky sessions). ➖ Adding/removing servers reshuffles hashing and breaks sessions.

# 12. Caching

**Caching** = store frequently accessed / precomputed data in a fast layer so future reads skip the slow path.

Example: 500 ms Mongo fetch + 100 ms compute = 600 ms. Cache the computed result in Redis → ~60 ms.

**Cache invalidation**: when underlying data changes, stale cache must go. Common technique: **TTL (time to live)** — e.g., expire after 24h; the first request after expiry repopulates from the DB. Alternative: **write-through** — on every DB write, also write the cache (e.g., contest leaderboard updated in DB and Redis simultaneously so rankings served from Redis stay current).

**Terms**: **cache hit** = data present; **cache miss** = absent, fall through to DB.

**Benefits**: lower latency; reduced backend/DB load; lower network & compute cost; better handling of high traffic.

### Types of cache

1. **Client-side** — browser cache (HTML/CSS/JS). Cuts requests and bandwidth.
2. **Server-side** — in-memory stores: Redis, Memcached.
3. **CDN cache** — static content (HTML/CSS/PNG/MP4) on geo-distributed edges: CloudFront, Cloudflare.
4. **Application-level** — embedded in app code; caches intermediate results or query results.

### Redis Deep Dive

- **In-memory** data-structure store — data in RAM, so orders of magnitude faster than disk-backed DBs.
- Why not use Redis for everything? RAM is small and expensive vs disk — too much data causes memory exhaustion.

- **Key-value** model. Values can be strings, lists, sets, hashes, sorted sets, etc.
- Run locally: `docker run -d --name redis-stack -p 6379:6379 -p 8001:8001 redis/redis-stack:latest`
- **Key naming convention**: colon-separated namespaces — `user:1`, `user:2:email`.

**String commands**

- `SET key value` — set
- `GET key` — read
- `SET key value NX` — set only if key doesn't exist
- `MGET key1 key2 …` — multi-get in one round trip

**List commands**

- `LPUSH key value` / `RPUSH key value` — push left / right
- `LPOP key` / `RPOP key` — pop left / right
- `LLEN key` — length
- **Queue (FIFO)** = LPUSH + RPOP. **Stack (LIFO)** = LPUSH + LPOP.

Clients: Node `ioredis`, Django `django-redis`, Go `redis.uptrace.dev`. Docs: redis.io/docs

# 13. Blob Storage

Text/numbers fit in rows and columns; files (mp4, png, jpeg, pdf) don't. A file as raw binary = a **Blob (Binary Large Object)**.

Why not store blobs in MySQL/MongoDB: blobs are huge (a video can be 1 GB), queries slow to a crawl, and you'd have to solve scaling/backup/availability for that volume yourself.

Instead use **managed blob storage**: **AWS S3**, Cloudflare R2. (Managed = provider handles scaling/security; you treat it as a black box.)

## AWS S3

Think "Google Drive for your app." Stores any file type. **Far cheaper per GB than RDS.**

Features: **scalability** (auto-scales to huge volumes), **durability** 99.999999999% (11 nines), **high availability** with SLAs, **pay-as-you-go cost**, **security** (encryption at rest and in transit, bucket policies, IAM), **access control** (policies, ACLs, pre-signed URLs).

# 14. Content Delivery Network (CDN)

Easiest way to scale **static** files. Serving an India-hosted S3 file to a user in the USA is slow; nearest-location serving is always faster.

A CDN is a fleet of geo-distributed servers caching and serving static content (images, video, stylesheets), cutting latency, load times, and bandwidth cost. Examples: **AWS CloudFront, Cloudflare CDN**.

**How it works**: user's request hits the nearest **edge server**. Hit → served immediately. Miss → edge fetches from the **origin server** (e.g., S3), caches it, then returns it. Subsequent requests are served from

the edge.

**Key concepts**

- **Edge servers** — geo-distributed cache nodes; users routed to the nearest.
- **Origin server** — your source of truth (S3 / web server).
- **Caching** — copies of static content held at the edge, cutting origin requests.
- **TTL** — how long a file stays cached at the edge before refresh (e.g., 24 h).
- **GeoDNS** — routes users to the nearest edge based on geography.

# 15. Message Broker

**Synchronous programming**: client requests → server processes → immediate response. Most apps work this way.

Breaks down for long tasks (10 minutes): the client can't wait and HTTP will time out. **Asynchronous**: immediately reply "your task is processing," do the work in the background, notify later (email/push).

Don't hand work to workers directly — put a **message broker** in between. The broker acts as a queue: the **producer** puts a task message in; the **consumer/worker** pulls it, processes it, and (for queues) deletes it.

**Why a broker in the middle:**

1. **Reliability** — producer can die; workers keep draining the queue.
2. **Retries** — if processing fails, the message is still in the broker and can be retried.
3. **Decoupling** — producer and consumer work at their own pace, independent of each other.

## Two types

| | **Message Queue** | **Message Stream** |
|---|---|---|
| Examples | RabbitMQ, AWS SQS | Apache Kafka, AWS Kinesis |
| Consumers per message | **One kind** of consumer | **Many kinds** of consumers |
| Deletion | Consumer deletes after processing (API provided) | Messages are **never deleted by consumers**; retained (manual delete or expiry) |
| Model | Pull-and-remove | Consumers **iterate** over messages independently |

**Message queue example**: video metadata → transcoder service converts to 480p/720p → deletes the message. Scale horizontally: any free consumer picks up a message → parallel processing (3 consumers = 3 messages at once).

**Why streams exist**: suppose you also want a Caption Generator for each video. With queues you'd need two queues and the producer writing to both — if it writes to one and crashes before the second, you get **inconsistency** (transcoded but no captions). A stream fixes this: **write once, read by many**. Each consumer type iterates independently over the same messages, each processing every message exactly once from its own position.

### When to use a message broker

Two microservices commonly talk via **REST API** or **message broker**. Use a broker when:

- The task is **non-critical** and can tolerate delay — e.g., sending email.
- The task is **long-running / compute-heavy** — e.g., video transcoding, PDF generation.

## 16. Apache Kafka Deep Dive

Kafka is a **message stream** with **very high throughput** — you can dump enormous volumes into it without it falling over.

**Classic use**: Uber tracking driver locations every 2 s. Thousands of writes every 2 s would crush a DB (low write throughput). Write them into Kafka instead; a consumer drains them in bulk every 10 minutes and batch-writes to the DB. DB work happens every 10 min instead of every 2 s.

### Internals

- **Producer** — publishes messages, e.g., `{"email", "message"}`.
- **Consumer** — subscribes to topics and processes the feed.
- **Broker** — the Kafka server storing and managing topics.
- **Topic** — a named category/feed ( `sendEmail` , `writeLocationToDB` ).
- Analogy: **Broker = DB server, Topic = table**.
- **Partition** — each topic is split into partitions for parallelism (analogous to sharding a table). You choose the partitioning scheme, e.g., `sendNotification` partitioned by region: North India → Partition 1, South India → Partition 2.
- **Consumer Group** — every consumer must belong to a group. Each group represents one *kind* of processing over a subset of partitions. E.g., group A = video transcoding, group B = caption generation, both reading the same topic.

**Rebalancing rules** (Kafka does this automatically, no code required):

- 4 partitions, 1 group with 3 consumers → C1: P0, C2: P1+P2, C3: P3.
- **One partition is consumed by exactly one consumer within a group.**
- **Different groups can each consume the same topic in full.**
- If consumers in a group **outnumber** partitions, the extras sit idle.
- ⇒ **To scale consumers horizontally, you must have at least as many partitions as consumers.**

## 17. Realtime Pub/Sub

- **Message broker**: consumer **pulls** the message (via API/SDK); the message waits in the broker until pulled.
- **Pub/Sub**: broker **pushes** the message to all subscribers the instant it's published — no polling, no API call.

Crucially, **pub/sub brokers do not retain messages**. Publish → fan out to current subscribers → done. Nothing stored.

Example broker: **Redis** (Redis is both a cache and a real-time pub/sub broker).

**Use case — horizontally scaled chat over WebSockets**: Client-1 is connected to Server-1, Client-3 to Server-2. Server-1 can't deliver directly to Client-3. Server-1 publishes to a Redis channel; Server-2 (subscribed) receives it instantly and pushes down its WebSocket to Client-3. Use pub/sub whenever you need very low latency fan-out.

# 18. Event-Driven Architecture (EDA)

**Motivating problem**: e-commerce checkout. Order Service → Payment Service → success page. But before responding, it also synchronously calls Inventory Service (decrement stock) and Notification Service (email). Neither contributes to the response, yet the client waits for both.

**EDA fix**: Order Service publishes an "order succeeded" **event** to a message broker and forgets about it. Inventory and Notification consume asynchronously. Client isn't blocked.

**Why EDA:**

1. **Decoupling** — producers don't know consumers. Without EDA, Inventory Service going down can break checkout since Order calls it directly. With EDA, Order doesn't care.
2. **Resilience** — one component failing doesn't block others.
3. **Scalability** — each service scales horizontally and independently.

### Four EDA patterns (first two dominate in practice)

1. **Simple Event Notification** — producer publishes only that something happened, with a lightweight payload (e.g., just `order_id`). Consumers query the DB for details they need.
2. **Event-Carried State Transfer** — producer publishes the **full state** needed, so consumers make no extra calls.
   - ➕ Lower latency (no follow-up network/DB calls).
   - ➖ Larger events → more broker storage and bandwidth cost.
3. **Event Sourcing** — (niche; not covered)
4. **Event Sourcing with CQRS** — (niche; not covered)

# 19. Distributed Systems

Summing primes to $10^{100}$ won't finish on one machine. A **distributed system** splits the work across many machines ($0 \to 10^{10}$ on one, $10^{10}+1 \to 10^{20}$ on the next, ...) and combines results.

Simply: **work done by a set of machines rather than one**. Sharding is a distributed system (table split across servers). Horizontal scaling is a distributed system (many servers serving requests).

**Hard parts**: coordinating machines, dividing work evenly. **From the client's perspective the system must look like a single machine** — distribution is an internal concern.

**CAP theorem is the fundamental law of all distributed systems.**

### Leader–Follower pattern

The common implementation: elect one server as **leader**; the rest are **followers**. Leader takes client requests, assigns work to followers, collects and combines their results, returns to client.

Leader must be chosen in two situations:

1. At system startup.
2. When the leader dies — followers must **quickly detect** the failure and elect a new leader.

**Leader election algorithms** (treat as a black box for interviews):

| Algorithm | Complexity |
|---|---|
| LCR | $O(N^2)$ |
| HS | O(N log N) |
| Bully | O(N) |
| Gossip Protocol | O(log N) |

**Distributed databases**: Cassandra, DynamoDB, MongoDB, Google Spanner, Redis — all use sharding + horizontal scaling for large volumes and efficient querying.

## 20. Auto-Recoverable Systems via Leader Election

Goal: always keep ≥ N healthy servers without a human watching dashboards.

Introduce an **Orchestrator** whose job is to monitor servers and restart any that die.

But: *who watches the orchestrator?* Answer: run **multiple orchestrators**, and use **leader election** to pick a **leader-orchestrator**.

Resulting self-healing hierarchy:

- Worker-orchestrators monitor and restart **application servers**.
- Leader-orchestrator monitors and restarts **worker-orchestrators**.
- If the **leader-orchestrator** dies, a worker-orchestrator promotes itself via leader election.

Net effect: **zero human intervention**.

## 21. Big Data Tools

Example: **Apache Spark** (also Flink). Big data tools are distributed systems for processing datasets too large for one machine.

Architecture: a **coordinator** + **workers**. Client → coordinator → splits the dataset into chunks → assigns to workers → collects and combines partial results → returns final answer.

The coordinator must handle:

- Reassigning a crashed worker's data to another machine
- **Recovery** — restarting downed workers

- Combining worker results
- Scaling and data redistribution
- Logging

**When to use**: when a single instance can't hold or process the data — training ML models, analyzing social networks / recommendation systems, ingesting from many sources into a warehouse.

Building this yourself is very hard, so use Spark/Flink as a black box: you supply only **business logic as jobs** (Python/Java/Scala) and the framework distributes execution.

# 22. Consistency Deep Dive

Consistency only matters for **distributed + stateful** systems.

- **Stateful**: machine stores data for later use → databases.
- **Stateless**: machine holds no meaningful data → most application servers.

Hence consistency discussions almost always concern databases. Always check what consistency model your DB (DynamoDB, Cassandra, MongoDB…) actually offers.

## Strong Consistency

- Any read after a write returns the **most recent** write.
- Once a write is acknowledged, **all** subsequent reads reflect it.
- All replicas must agree before a write is acknowledged.
- The system behaves as if there were **one single copy** of the data.

**Choose when**: bank transfers, trading apps needing correct latest share prices.

## Eventual Consistency

- No immediate guarantee; replicas converge **after some time**.
- Reads may briefly return stale data.
- Trading consistency buys **high availability** (CAP).
- Requires a **conflict resolution mechanism** when replicas diverge.

**Choose when**: social media ( `likeCount` correcting later is fine), e-commerce product catalogs.

## Achieving strong consistency

1. **Synchronous replication** — all replicas updated before ack. E.g., Google Spanner.
2. **Quorum-based protocols** — in leader-follower setups, followers ack reads/writes.
   - *Read quorum (R)* = number of nodes returning data for a read.
   - *Write quorum (W)* = number of nodes acking a write.
   - **Strong consistency requires** `W + R > N` (N = total nodes). Used by DynamoDB, Cassandra.
3. **Consensus algorithms** — leader election + a write/read succeeds when >50% of nodes ack. Learn **Raft** (the approachable one); Docker Swarm uses Raft internally.

### Achieving eventual consistency

1. **Asynchronous replication** — ack immediately, propagate in the background. Default for Cassandra, MongoDB, DynamoDB.
2. **Relaxed quorum** — `R + W ≤ N`. E.g., DynamoDB eventual-consistency mode.
3. **Vector clocks** — versioning to order/merge concurrent updates. E.g., DynamoDB.
4. **Gossip protocol** — nodes exchange **heartbeats** (periodic HTTP/TCP pings every 2–3 s) with a random subset, spreading updates and detecting failed nodes. Used by DynamoDB and Cassandra.

## 23. Consistent Hashing

An **algorithm that decides which key belongs to which node.** Nothing more. Used in distributed **stateful** systems — i.e., databases. (You'd use it when building a database, not a typical backend.)

### The problem with naive hashing

```
server_for(key) = HASH(key) % num_servers
```

Fine while `num_servers` is fixed. But with auto-scaling, going 3 → 2 servers changes `HASH(key) % N` for **most keys**, forcing massive data movement.

### How consistent hashing works

1. Hash each **server identity** (IP/ID) with e.g. SHA-128 → a number in `[0, 2¹²⁸)`.
2. Visualize a **ring** spanning `[0, 2¹²⁸)`; place each server at its hash position.
3. Hash each **key** with the same function; place it on the ring too.
4. **Each key belongs to the first server clockwise from it.**

Example placement: key-1, key-4 → Node-1; key-3 → Node-2; key-2, key-5 → Node-3.

**Payoff**: remove Node-2 and only key-3 moves (to Node-1). Everything else is untouched — **minimal key movement** on topology change.

Note: consistent hashing only *tells you* the mapping; **you** must perform the actual data movement (snapshots/copies).

Used internally by: **DynamoDB, Cassandra, Riak**.

## 24. Data Redundancy and Recovery

**Redundancy** = keeping multiple copies of the same data on multiple DB servers.

**Why:**

- Survive physical disaster (data center flood/fire) without data loss.
- Survive technical failure (disk corruption/crash) without data loss.

**Backup approaches** (no universal rule — a preference):

- **Daily snapshot** at night, replicated to a different DB server.
- **Weekly backup.**

- Worst case, you lose at most a day's / a week's data.

### Continuous Redundancy (modern standard)

Two DB servers: a **main** and a **replica**.

- All client reads/writes go to the **main**.
- Changes replicate to the replica **synchronously or asynchronously** (your config).
- The replica serves **no client traffic** — its only job is staying in sync.
- On main failure, the **replica is promoted to main** and starts serving.

Result: a resilient system with near-zero data loss.

## 25. Proxy

A **proxy** is an intermediary server sitting between a client and another server.

### Forward Proxy — acts on behalf of the client

The destination server sees only the proxy's IP, never the client's. **A VPN is a forward proxy.**

**Main feature: hides the client.**

Use cases:

- Access restricted/geo-blocked content.
- **Caching** frequently accessed content so the client never reaches the origin.
- Organizations filtering/controlling employee internet usage.

Flow: client → forward proxy → server → forward proxy → client.

(System design focuses far more on reverse proxies, since forward proxies are a client-side concern.)

### Reverse Proxy — acts on behalf of the server

Clients send requests to the reverse proxy, which routes them to the right backend server and returns the response. The client never learns the real server.

**Main feature: hides the server.** A **load balancer is a reverse proxy**.

Use cases:

- **Load balancing**
- **SSL termination** — handle encryption/decryption so backends don't have to
- **Caching** static content
- **Security** — backends are never directly exposed to the internet

Examples: **Nginx**, **HAProxy**.

### Minimal reverse proxy in Node.js

Routes `/product` → `localhost:5001`, `/order` → `localhost:5002`.

```
npm init -y
npm install http-proxy
```

```
const http = require('http');
const httpProxy = require('http-proxy');

const proxy = httpProxy.createProxyServer();

const targets = {
  productService: 'http://localhost:5001',
  orderService: 'http://localhost:5002',
};

const server = http.createServer((req, res) => {
  if (req.url.startsWith('/product')) {
    proxy.web(req, res, { target: targets.productService }, (error) => {
      console.error('Error proxying to product service:', error.message);
      res.writeHead(502);
      res.end('Bad Gateway');
    });
  } else if (req.url.startsWith('/order')) {
    proxy.web(req, res, { target: targets.orderService }, (error) => {
      console.error('Error proxying to order service:', error.message);
      res.writeHead(502);
      res.end('Bad Gateway');
    });
  } else {
    res.writeHead(404);
    res.end('Route not found');
  }
});

const PORT = 8080;
server.listen(PORT, () => {
  console.log(`Reverse proxy server running at http://localhost:${PORT}`);
});
```

In production, don't hand-roll it — use **Nginx**, which is highly optimized and adds caching, SSL termination, etc. Same principle applies to load balancers, message brokers, Redis: use battle-tested tools as black boxes rather than reinventing them.

## 26. How to Solve Any System Design Problem

1. **Understand the problem statement** — clarify requirements and which features are in scope (e.g., "build Amazon" → which features exactly?).
2. **Break the big problem into small solvable sub-problems** — for e-commerce: product listing, product search, orders, payment handling.

3. **Solve each sub-problem against four axes:**
   - **Database** (SQL vs NoSQL, schema, sharding/replication)
   - **Caching** (what, where, invalidation)
   - **Scaling & fault tolerance** (horizontal/vertical, LB, redundancy)
   - **Communication** (sync REST vs async message broker / EDA)
4. Recurse: split a sub-problem further only **when genuinely needed** — don't over-complicate the design.

## Quick Decision Cheat Sheet

| Situation | Reach for |
|---|---|
| Slow reads on one table | Indexing → partitioning |
| Read-heavy traffic | Master-slave replication, caching (Redis), CDN |
| Write-heavy traffic | Sharding, multi-master, Kafka buffering + batch writes |
| Money / correctness critical | SQL, ACID, strong consistency, CP |
| Social feed / catalog | NoSQL, eventual consistency, AP |
| Long-running task | Async + message broker (queue) + workers |
| One event, many independent consumers | Message stream (Kafka), EDA |
| Instant fan-out, no retention needed | Realtime pub/sub (Redis) |
| Files (images, video, PDF) | Blob storage (S3) + CDN in front |
| Traffic varies by time of day | Auto scaling behind a load balancer |
| Nodes join/leave frequently in a data cluster | Consistent hashing |
| Need self-healing infrastructure | Orchestrators + leader election |
| Dataset too big for one machine to process | Spark / Flink (coordinator + workers) |
| Hide backends, terminate SSL, route by path | Reverse proxy (Nginx / HAProxy) |